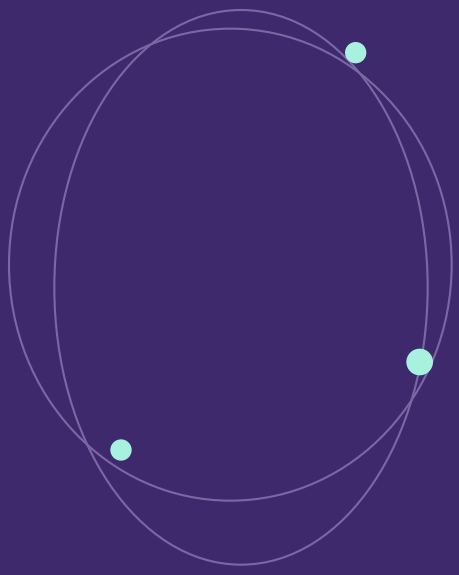
DISPOSITIVOS MÓVILES / ENTREGA FINAL

Caña Dulce

Manual técnico

Arquitectura, patrones y operación



Android 1.2.3 / main · 1493c59

03	23	06
Capas del sistema	Rutas HTTP	Tablas de MySQL

Referencia de la versión de distribución de Caña Dulce. Describe cómo el cliente Android se comunica con la API publicada y cómo se mantienen sus datos y reglas.

Faber Enrique Uni Gomez

Proyecto Caña Dulce · Manual de la entrega final

MANUAL TÉCNICO

Índice

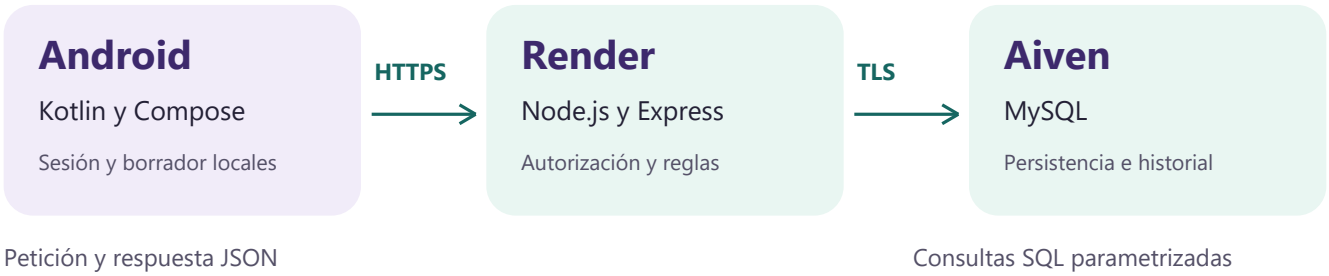
Los títulos del índice abren sus secciones. Los enlaces al código apuntan a la revisión documentada.

Pág.	Sección
03	Cómo se conectan las tres capas
04	Versiones y estructura
05	Estado y presentación en Android
06	Capas de la API y seguridad
07	Modelo relacional e integridad
08	Acceso, estudiantes y lecturas
09	Cuestionarios, intentos y borrado
10	De una respuesta al gesto
11	Calificación y gamificación
12	Preparar el backend de pruebas
13	Desplegar y mantener la API
14	Compilar, firmar e instalar Android
15	Pruebas, evidencias y alcance

01 · ARQUITECTURA

Cómo se conectan las tres capas

Caña Dulce separa la interfaz Android, las reglas de negocio en Express y la persistencia relacional en MySQL.



Capa	Responsabilidad
Android	Compone las pantallas, administra el estado del usuario y envía peticiones JSON. Conserva la sesión cifrada, preferencias y borrador en el dispositivo.
API en Render	Recibe las peticiones HTTPS. Verifica identidad, permisos y datos. Aplica las reglas de lecturas, cuestionarios, calificación y progreso.
MySQL en Aiven	Almacena perfiles, lecturas, preguntas, opciones, intentos y respuestas. Las claves y restricciones mantienen las relaciones entre ellos.

Dos conexiones independientes

- **Teléfono → Render.** `ApiCanaDulce` toma la URL base de `BuildConfig.API_URL`. Las operaciones privadas llevan un token Bearer y devuelven JSON o una respuesta vacía cuando corresponde.
- **Render → Aiven.** El proceso Node.js usa el pool de `db.js` con TLS verificado. Las credenciales de base de datos pertenecen al entorno del servidor.

Comprobación de la nube. GET /health ejecuta SELECT 1. HTTP 200 con baseDeDatos “conectada” confirma ambas capas del backend en ese momento.

Endpoint público de comprobación

backend/src/server.js · línea 19

02 · TECNOLOGÍAS

Versiones y estructura

La revisión de código es 1493c59 en main. La aplicación de distribución documentada es 1.2.3, código de versión 6 y paquete com.canadulce.app.

Componente	Versión o configuración	Uso
Kotlin y Compose	Kotlin 2.2.10, Compose BOM 2026.02.01, Material 3	Pantallas declarativas, estado visual, componentes y animación.
Android y Gradle	SDK de compilación y destino 37, mínimo 24, AGP 9.4.1, Gradle 9.6.0	El entorno documentado ejecuta Gradle con JDK 25. El módulo declara compatibilidad Java 11 para el código.
Node.js y Express	Node 24.x, Express 5.3.0	API REST organizada en routers, middleware y servicios.
MySQL y controlador	MySQL 8, mysql2 3.24.6	Consultas parametrizadas, pool y transacciones.
Sesión y criptografía	jsonwebtoken 9.0.3, Node crypto, Android Keystore	JWT, hash scrypt de códigos y almacenamiento cifrado de sesión.
Herramientas	Postman, Compose UI Test y AndroidJUnitRunner	Pruebas HTTP y recorridos de la interfaz en emulador.

Carpetas que se usan en mantenimiento

- `android/app/src/main/` contiene el cliente Android. `android/app/src/androidTest/` contiene pruebas instrumentadas.
- `backend/src/` contiene el servidor. `backend/database/` contiene el esquema y el cargador de misiones.
- `docs/` guarda el contrato, la colección de Postman y el registro de validación. `render.yaml` define el despliegue.

android/gradle/libs.versions.toml

android/app/build.gradle.kts

backend/package-lock.json

03 · PATRONES DE DESARROLLO

Estado y presentación en Android

La separación de responsabilidades sigue un esquema de presentación con ViewModel. Compose dibuja el estado observable y envía las acciones del usuario al ViewModel.

Pieza	Implementación y función
Entrada y navegación por estado	MainActivity obtiene PrincipalViewModel y monta AplicacionCanaDulce. La función raíz selecciona el destino a partir de EstadoApp.pantalla.
Flujo de estado y eventos	La pantalla envía un evento. PrincipalViewModel valida o pide datos, crea una copia de EstadoApp con copy y Compose vuelve a dibujar las partes que cambiaron.
Cliente HTTP centralizado	ApiCanaDulce.solicitar arma la URL, encabezados y JSON. Usa HttpURLConnection en Dispatchers.IO, espera hasta 20 s al conectar y 90 s al leer. Adapta errores a ErrorApi.
Modelos de intercambio	Perfil, Lectura, Pregunta, Intento y Resultado convierten JSON en objetos Kotlin. EstadoApp contiene los datos necesarios para la interfaz.
Componentes reutilizables	TarjetaMision, CampoCodigo y ReaccionRespuesta encapsulan partes compartidas. El tema reúne colores y tipografía.

Qué persiste en el dispositivo

AlmacenSesion guarda el token y el perfil cifrados con AES/GCM y una clave en Android Keystore. Al reabrir, el ViewModel consulta la sesión actual para comprobarla.

BorradorCuestionario guarda el paso y las opciones sin confirmar por perfil e intento. Al reanudar, la API devuelve las respuestas confirmadas. Las preferencias de personaje, tamaño de texto y movimiento se conservan localmente.

android/app/src/main/java/com/canadulce/app/ui/AplicacionCanaDulce.kt · línea 16

android/app/src/main/java/com/canadulce/app/ui/PrincipalViewModel.kt · línea 162

android/app/src/main/java/com/canadulce/app/datos/ApiCanaDulce.kt · línea 14

android/app/src/main/java/com/canadulce/app/datos/AlmacenSesion.kt

android/app/src/main/java/com/canadulce/app/datos/BorradorCuestionario.kt

04 · PATRONES DE DESARROLLO

Capas de la API y seguridad

Patrón aplicado	Qué se ve en el código
Router y servicio	server.js monta seis routers. Las rutas reciben HTTP y aplican validación. servicios/intentos.js, lecturas.js, preguntas.js y progreso.js concentran reglas y consultas.
Middleware	autenticar extrae el Bearer y comprueba el token. Para estudiantes consulta su perfil vigente. permitirRol protege las operaciones que requieren un rol específico.
Consultas parametrizadas y pool	mysql2/promise reutiliza conexiones. Las consultas con execute separan valores del SQL con marcadores ?. El pool define hasta cinco conexiones y una cola de veinte.
Transacción por operación	ejecutarTransaccion usa READ COMMITTED, hace commit o rollback y devuelve o descarta la conexión. Las operaciones concurrentes sensibles aplican bloqueos de filas y restricciones.

Identidad, autorización y credenciales

- Los códigos de estudiantes se almacenan como **hash scrypt con sal aleatoria**. La contraseña docente se verifica contra un hash configurado en el entorno.
- El JWT usa **HS256**, expira en **8 horas** y verifica emisor y audiencia. El token contiene el rol y el identificador del usuario. Cerrar sesión en Android elimina su copia local.
- El servidor verifica **rol, propiedad y grado** en las operaciones que lo requieren. En el cuestionario estudiantil la clave de cada opción se omite.
- Las variables privadas se configuran en el entorno. Para MySQL remoto, `db.js` verifica la CA y la identidad del servidor.

backend/src/middleware/autenticacion.js · línea 4

backend/src/tokens.js

backend/src/seguridad.js · línea 13

backend/src/transacciones.js · línea 3

backend/src/db.js · línea 46

05 · BASE DE DATOS

Modelo relacional e integridad

Las flechas muestran relaciones de uno a muchos. RESPUESTA relaciona el intento con una pregunta y con la opción elegida.

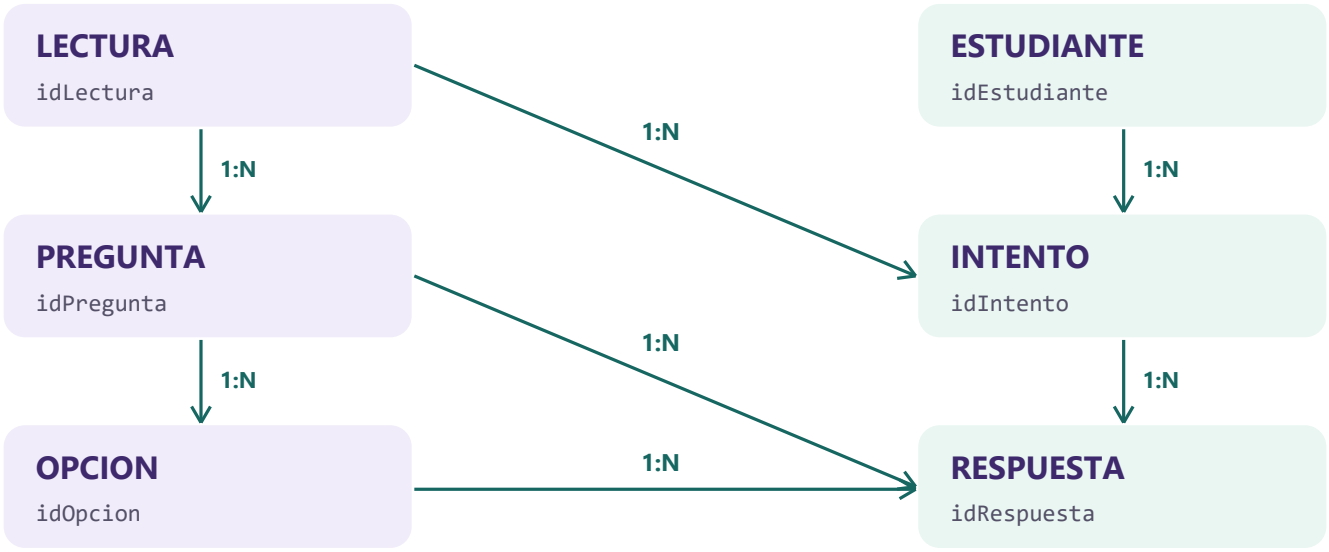


Tabla	Datos clave y regla
`ESTUDIANTE`	Alias normalizado único, grado 3 o 4, estado y hash del código.
`LECTURA`	Texto, grado, dificultad, categoría y estado de publicación.
`PREGUNTA`	Pertenece a una lectura. Guarda enunciado, tipo, orden único por lectura y estado.
`OPCION`	Pertenece a una pregunta. Guarda texto, posición y marca de corrección.
`INTENTO`	Vincula estudiante y lectura. Almacena total de preguntas, estado, fechas y puntaje.
`RESPUESTA`	Guarda pregunta, opción y corrección. La clave compuesta impide asociar una opción de otra pregunta.

- La columna generada `pendiente` y su índice único permiten **un intento en curso por estudiante y lectura**.
- `RESPUESTA` tiene una restricción única por intento y pregunta. El servicio bloquea la edición o eliminación de contenido con intentos para conservar el historial.

backend/database/schema.sql · línea 125

backend/database/schema.sql · línea 192

06 · CONTRATO HTTP

Acceso, estudiantes y lecturas

La API expone 21 operaciones de negocio y dos rutas auxiliares. Su base pública es https://cana-dulce-api.onrender.com. Usa JSON UTF-8 e identificadores enteros positivos. Para rutas privadas, envía Authorization: Bearer <token>.

Método y ruta	Acceso y respuesta normal
`GET /health`	**Público** · 200, estado de API y MySQL
`POST /api/estudiantes`	**Público** · 201, perfil sin código ni hash
`POST /api/sesiones/estudiante`	**Público** · 200, token, rol y perfil
`POST /api/sesiones/docente`	**Público** · 200, token y rol
`GET /api/sesiones/actual`	**Autenticado** · 200, rol y perfil cuando corresponde
`GET /api/estudiantes`	**Docente** · 200, `{datos,total}`
`GET /api/estudiantes/:id/progreso`	**Propietario o docente** · 200, resumen e insignias
`GET /api/lecturas`	**Autenticado** · 200, `{datos,total}`
`GET /api/lecturas/:id`	**Autenticado con acceso** · 200, lectura con contenido
`POST /api/lecturas`	**Docente** · 201, lectura INACTIVA
`PUT /api/lecturas/:id`	**Docente** · 200, lectura actualizada
`DELETE /api/lecturas/:id`	**Docente** · 204, desactivación lógica

Reglas de consulta

- En el listado estudiantil, la API deriva el grado del perfil autenticado y muestra las misiones activas y publicables de ese grado.
- El docente puede buscar perfiles por alias y filtrar por grado o estado. Consultar el progreso de un estudiante requiere ser el dueño del perfil o tener rol docente.

docs/API.md

docs/CanaDulce.postman_collection.json

07 · CONTRATO HTTP

Cuestionarios, intentos y borrado

Método y ruta	Acceso y respuesta normal
`POST /api/lecturas/:id/preguntas`	**Docente** · 201, pregunta y opciones
`GET /api/lecturas/:id/preguntas`	**Autenticado con acceso** · 200, preguntas sin claves
`GET /api/docente/lecturas/:id/preguntas`	**Docente** · 200, preguntas con claves
`PUT /api/preguntas/:id`	**Docente** · 200, pregunta actualizada
`DELETE /api/preguntas/:id`	**Docente** · 204, desactivación lógica
`POST /api/intentos`	**Estudiante** · 201, intento EN_PROGRESO
`GET /api/intentos`	**Propietario o docente** · 200, `{datos,total}`
`GET /api/intentos/:id`	**Propietario o docente** · 200, intento y resultado si existe
`POST /api/intentos/:id/respuestas`	**Estudiante propietario** · 200, respuesta confirmada y `esCorrecta`
`POST /api/intentos/:id/finalizar`	**Estudiante propietario** · 200, resultado y respuestas
`DELETE /api/lecturas/:id/eliminar`	**Docente** · 204, eliminación de una lectura sin intentos

Estado	Cuándo se usa
400 / 413	Entrada inválida o cuerpo superior al límite del parser de JSON (64 KB).
401 / 403	Falta una sesión válida, o el rol o propietario no tiene permiso.
404 / 409	Recurso inexistente o no disponible, o conflicto con el estado actual.
500 / 503	Error interno o indisponibilidad temporal. La respuesta no expone el SQL ni credenciales.

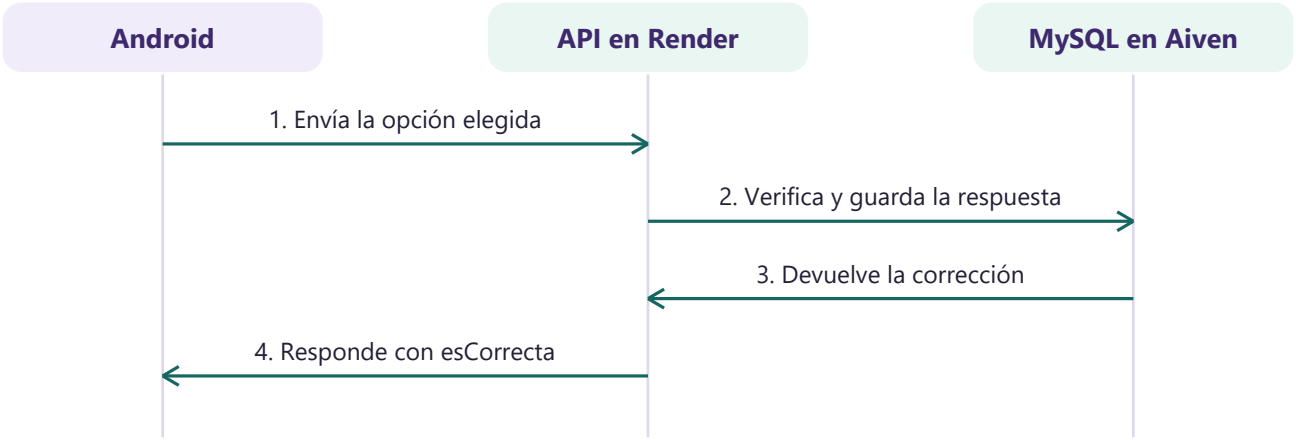
Diferencia operativa. DELETE /api/lecturas/:id desactiva y conserva el historial. DELETE /api/lecturas/:id/eliminar borra la lectura y sus preguntas y opciones si todavía no existen intentos.

backend/src/servicios/lecturas.js · línea 198

08 · FLUJO DE PUNTA A PUNTA

De una respuesta al gesto

Ejemplo con identificadores ficticios. El estudiante selecciona una opción y la app solicita al servidor que confirme el resultado.



Petición y respuesta de ejemplo

```
POST /api/intentos/12/respuestas
Authorization: Bearer <token>
Content-Type: application/json

{"idPregunta": 7, "idOpcion": 21}

HTTP 200
{"idPregunta": 7, "idOpcion": 21, "esCorrecta": true}
```

En Express. Se valida la sesión, el propietario del intento y que la opción pertenezca a una pregunta activa de esa lectura. Se bloquea el intento para guardar la respuesta de forma consistente.

Al volver a Android. El ViewModel actualiza el estado. `ReaccionRespuesta` activa `CELEBRAR` si `esCorrecta` es verdadero o `DESANIMAR` si es falso. Después del gesto, el estudiante puede continuar.

> **Reintento de red.** La misma opción ya confirmada devuelve la respuesta guardada. Intentar cambiar la opción devuelve HTTP 409.

android/app/src/main/java/com/canadulce/app/ui/PrincipalViewModel.kt · línea 348

backend/src/servicios/intentos.js · línea 108

09 · REGLAS DEL DOMINIO

Calificación y gamificación

Regla	Implementación
Finalizar un intento	Se exige una opción válida por pregunta activa. Se conservan las respuestas ya confirmadas y se guarda la calificación en una transacción. Reenviar la misma selección devuelve el resultado existente.
Puntaje	Aciertos × 100 ÷ total, redondeado a dos decimales. Lo calcula el servicio de intentos con las funciones de progreso.
Estrellas	0 puntos da 0 estrellas. Más de 0 y menos de 50 da 1. Desde 50 y menos de 80 da 2. Desde 80 da 3.
Insignias	Primera aventura con una lectura distinta completada. Explorador de historias con tres. Lectura brillante con 100 puntos.

Animaciones en el cliente

- **Espacio.kt** dibuja Astro y Lina con `Canvas`. `rememberInfiniteTransition` crea la flotación. `Animatable` y `tween` controlan los gestos de celebración y desánimo.
- **ReaccionRespuesta.kt** elige el gesto según la corrección devuelta por la API. Con el movimiento pausado, quedan la postura y el mensaje.
- **DespegueLunar.kt** anima la trayectoria del cohete con Compose. La pantalla de resultado muestra esta celebración para el estudiante si acertó todas las preguntas.
- **PantallaEntrenamiento.kt** contiene tres retos locales para practicar. El puntaje y las insignias se calculan a partir de intentos evaluados por el servidor.

backend/src/servicios/progreso.js · línea 3

android/app/src/main/java/com/canadulce/app/ui/componentes/Espacio.kt · línea 117

android/app/src/main/java/com/canadulce/app/ui/componentes/DespegueLunar.kt · línea 35

10 · OPERACIÓN

Preparar el backend de pruebas

Para trabajar en un entorno independiente, instala Node.js 24 y MySQL 8. Usa una base propia o una instancia de pruebas con acceso autorizado.

1. Clona el repositorio y abre `backend`. En MySQL Workbench crea y selecciona una base vacía. Ejecuta `database/schema.sql` con una cuenta que pueda crear tablas. Configura un usuario de aplicación con los permisos de lectura y escritura requeridos en ese esquema.
2. Ejecuta `npm ci`. En la primera configuración, copia `.env.example` como `.env`. Reemplaza sus datos de conexión por los de tu base de pruebas.

Grupo	Variables y valor esperado
MySQL	DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD. Para MySQL remoto usa DB_SSL=true y DB_SSL_CA_PATH con la CA del servicio. En un MySQL local aislado sin TLS, usa DB_SSL=false en desarrollo.
Sesión	JWT_SECRET debe contener 64 caracteres hexadecimales. Puedes generarlos con node:crypto usando randomBytes(32).toString('hex').
Docente	DOCENTE_USUARIO identifica la cuenta. DOCENTE_PASSWORD_HASH contiene el resultado de crearHashCodigo de seguridad.js. Genera el hash en una consola privada y guárdalo en el entorno del backend.

3. Inicia Express y comprueba la conexión. Si quieres cargar la biblioteca espacial, ejecuta el cargador sobre la base de pruebas ya configurada.

```
# Desde la carpeta backend
npm start

# En otra terminal
curl.exe -i http://localhost:3000/health
node --env-file=.env database/cargar-misiones.js
```

Biblioteca inicial. El cargador usa una transacción y omite lecturas ya existentes por título y grado. Para recargar otros datos, prepara una base separada.

backend/.env.example

backend/database/cargar-misiones.js · línea 15

11 · OPERACIÓN

Desplegar y mantener la API

Ajuste de Render	Valor del proyecto
Servicio	Web Service con entorno Node y rama main. Directorio raíz backend.
Construcción e inicio	npm ci --omit=dev y npm start. Render asigna PORT.
Comprobación	/health. Devuelve 503 si el servidor arranca y MySQL no responde.
Entorno	NODE_ENV=production, DB_SSL=true y DB_SSL_CA_PATH=certificados/aiven-ca.pem para Aiven. Configura también las variables DB_*, JWT_SECRET y la cuenta docente.
Actualización	Despliegue manual según render.yaml. En Render, usa Manual Deploy → Deploy latest commit para aplicar la revisión elegida.

Verificación después de publicar

1. Abre `/health` y verifica **HTTP 200** y `baseDeDatos: "conectada"`. Si falla, revisa los registros de Render y el estado de MySQL en Aiven.
2. Si hay un error de conexión, comprueba el host, el puerto, el usuario, el esquema y la ruta de la CA. Tras inactividad, el primer acceso del servicio gratuito puede tardar en iniciar.
3. Abre la app con Internet. Comprueba el ingreso con un perfil autorizado y la consulta de misiones. Para pruebas que escriben datos, usa una base de pruebas.

Respaldar y restaurar

Exporta el esquema y los datos con MySQL Workbench o `mysqldump`, mediante una cuenta autorizada y TLS. Restringe el acceso a la copia. Restaura primero en una base separada y verifica relaciones, conteos y `/health` antes de cambiar la conexión de producción.

render.yaml

Health check de Caña Dulce

12 · OPERACIÓN

Compilar, firmar e instalar Android

1. Abre la carpeta `android` en Android Studio con el JDK 25 configurado para Gradle y SDK 37. Sincroniza con el Gradle Wrapper del proyecto.
2. Desde PowerShell, compila el tipo de APK que necesites. El APK de distribución se genera sin firmar en `app/build/outputs/apk/release/app-release-unsigned.apk`.

```
# Desde la carpeta android
.\gradlew.bat :app:lintDebug :app:assembleDebug
.\gradlew.bat :app:assembleRelease
```

3. Para usar un backend local desde el emulador, genera una compilación de desarrollo. `10.0.2.2` apunta al equipo que ejecuta el emulador. El APK de distribución usa HTTPS en Render.

```
.\gradlew.bat :app:assembleDebug "-PapiLocal=http://10.0.2.2:3000/"
```

Firma de distribución

En una carpeta que contenga el APK sin firmar, alinea el archivo con `zipalign`, firma con el almacén de la entrega y comprueba la firma. Los comandos requieren las herramientas del SDK en el PATH. Configura la contraseña en el entorno de la sesión de firma.

```
$ks = 'C:\ruta\cana-dulce-release.p12'
zipalign -P 16 -f 4 app-release-unsigned.apk app-aligned.apk
apksigner sign --ks $ks --ks-key-alias cana-dulce `
  --ks-pass env:CD_FINAL_KEY_PASS --out Cana_Dulce.apk app-aligned.apk
apksigner verify --verbose --print-certs Cana_Dulce.apk
adb install Cana_Dulce.apk
```

Actualizaciones. Conserva la clave de distribución fuera del repositorio. Para actualizar una instalación firmada, usa el mismo paquete y la misma firma y aumenta el código de versión.

android/app/build.gradle.kts · línea 16

android/app/src/debug/AndroidManifest.xml

13 · VERIFICACIÓN

Pruebas, evidencias y alcance

Área	Qué quedó registrado
Backend	58 comprobaciones generales en MySQL temporal y 50 del flujo de confirmación. También se registraron 53 comprobaciones asociadas a la mejora de eliminación.
Android	Recorridos de acceso, reanudación, reacciones, resultado y gestión docente registrados en emulador. El APK firmado se verificó y arrancó en ese entorno.
Nube	La comprobación puntual de esta revisión obtuvo HTTP 200 en /health con baseDeDatos “conectada”. Verifica disponibilidad de Express y MySQL al momento de la consulta.
Validación con cliente	El acta firmada registra la conformidad y las tareas observadas. La evidencia final incluye las fotos de la orientación a la docente y de la actividad con estudiantes.

Cómo repetir las comprobaciones

- Importa la colección vigente de Postman desde `docs/`. Carga las credenciales de prueba como variables locales. Para repetir altas y cambios, usa un entorno aislado y un alias nuevo.
- En Android, usa `FlujosSprintUnoTest`, `FlujosSprintDosTest` y `ReaccionesPersonajeTest`. Los recorridos con acceso real requieren argumentos del ejecutor y la biblioteca de demostración. El procedimiento está en `docs/VALIDACION.md`.
- El resultado de `/health` es una comprobación de conectividad. Las aserciones de permisos, resultados e idempotencia se documentan por separado en las pruebas.

Identificación de la entrega. La validación técnica del 6 de noviembre registra 88202b1 como revisión funcional desplegada en ese momento. El manual documenta el código fuente 1493c59 y la versión de distribución 1.2.3.

docs/VALIDACION.md

docs/CanaDulce.postman_collection.json

Código de las pruebas Android